

Formal Architecture and Machinery of Appendices G, I, J, K, and L

An engineer-oriented guide for hardware architects and design-verification engineers

Stuart Swan, Platform Architect, Siemens EDA

24 July 2026

G Contract	I Local replay	J Safety glue	K Liveness	L Witness selection
----------------------	--------------------------	-------------------------	----------------------	-------------------------------

Based on: Catapult HLS User View Scheduling Rules, 24 July 2026

This guide is explanatory. The scheduling-rules document remains the normative source.

The latest version of the scheduling rules document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

Audience

Readers comfortable with state machines, handshakes, FIFOs, assertions, and basic formal reasoning, but not necessarily specialists in trace semantics or mechanized theorem proving.

Contents

- [1](#) Executive orientation
- [2](#) Architecture at a glance
- [3](#) Core vocabulary and proof boundaries
- [4](#) Appendix G - the observable scheduling contract
- [5](#) Appendix I - per-process replay and preparation
- [6](#) Appendix J - local-to-global safety construction
- [7](#) Appendix K - conditional deadlock preservation
- [8](#) Appendix L - witness selection and snooping
- [9](#) How the appendices work together in a verification flow
- [10](#) Assumptions and evidence matrix
- [11](#) Common misunderstandings
- [12](#) Quick-reference index

1. Executive orientation

PURPOSE

Provide a single mental model for five appendices that otherwise appear as separate layers of formal detail.

Engineer takeaway: Treat the appendices as a proof stack: G specifies the contract, I generates local evidence, J constructs the global safety witness, K proves a separate conditional deadlock result, and L selects source witnesses when latency-sensitive choices matter.

The formal architecture is best understood as two related, but deliberately separate, proof pipelines.

- **Safety pipeline (G → I → J).** Shows that a covered reachable RTL behavior has a matching behavior in the prescribed source reference model at the chosen observable boundary. This is an observable trace-compatibility claim, not cycle-by-cycle or full-state equivalence.
- **Liveness/deadlock pipeline (G → I → J → K).** Starts from the safety correspondence, reconstructs the wait-relevant cutpoint observation, and then rules out a specific class of reachable closed internal wait-cycle cutpoints under additional global progress and liveness premises.
- **Witness-selection adapter (L).** Supplies the WC witness needed by I and J when failed non-blocking polls or other latency-sensitive epsilon choices affect later observable behavior. It selects an admissible source execution aligned with the free-running RTL; it does not redefine the source model or stall the RTL.

KEY ARCHITECTURAL FACT: Safety and liveness are not one theorem

Appendix J establishes a global observable-safety correspondence from local evidence. Appendix K does not strengthen that correspondence into liveness automatically. It adds a different argument with different assumptions: wait-for-graph reconstruction, drain discipline, fairness/progress, cutpoint coverage, and the scheduler-robust source-liveness premise A5-L.

1.1 What the formal framework is trying to achieve

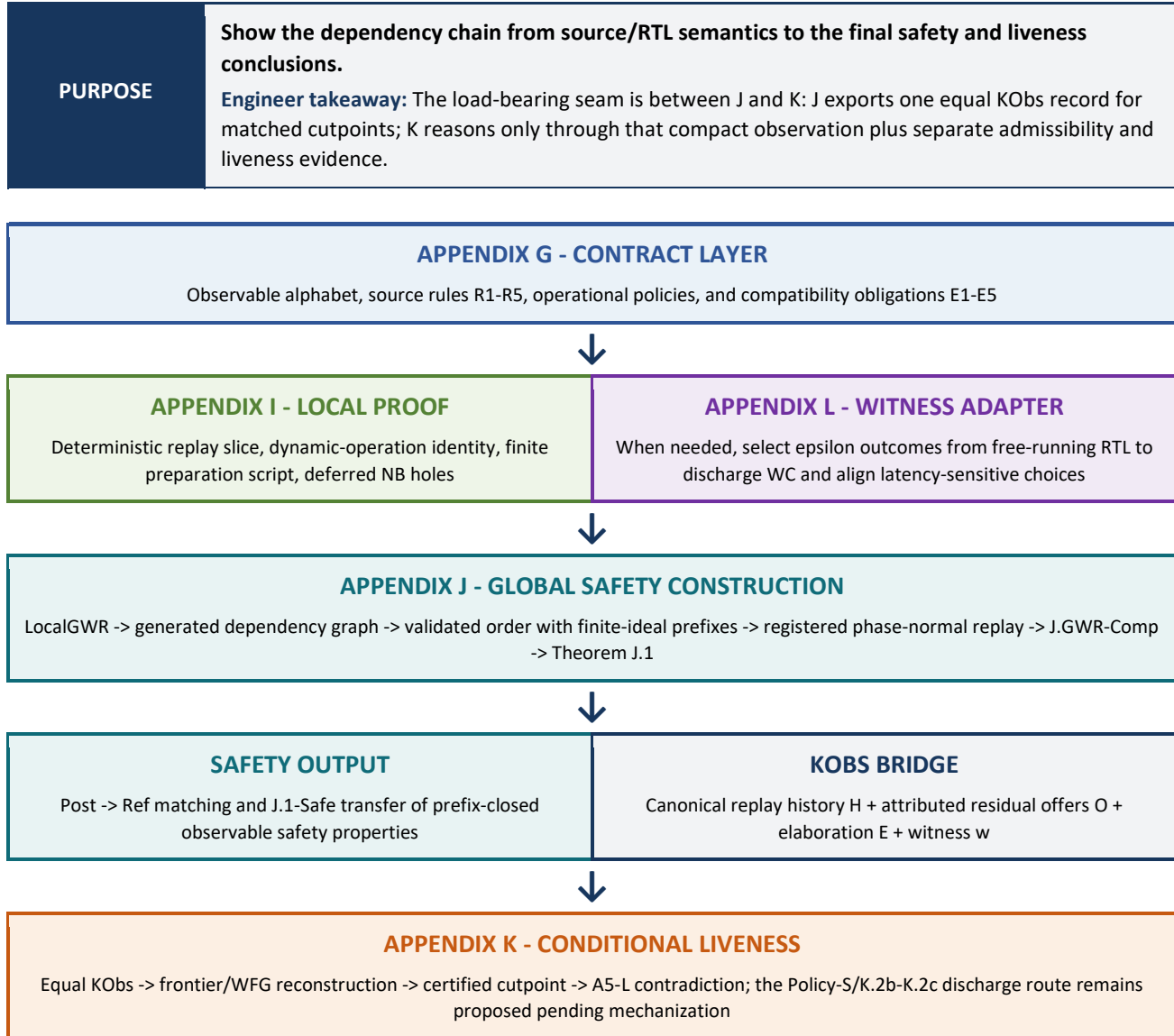
The user-facing goal is a verification flow in which most functional verification and debug can remain at the pre-HLS SystemC level, while the RTL is checked against a precise implementation contract. The formal appendices make that promise conditional and auditable: they identify exactly which source rules, implementation properties, witness choices, boundary abstractions, and liveness assumptions must hold.

The central idea is to compare only architecturally meaningful observations: committed message transfers, synchronization operations, and anchored signal reads/writes at selected boundaries. Internal FSM states, pipeline movement, arbitration bookkeeping, and other micro-steps are represented as epsilon behavior unless they affect a named observable or proof predicate.

1.2 What the framework does not claim

- It does not claim identical cycle timing between pre-HLS and post-HLS models.
- It does not claim equality of all internal state, requests, or pipeline registers.
- It does not automatically prove that a particular HLS run or RTL instance satisfies E3, E4, B-faithfulness, R.RegSeq, or the other implementation obligations.
- It does not infer global deadlock freedom from the source-order no-reverse rule alone.
- It does not treat an arbitrary finite Policy-S simulation prefix as a complete liveness proof.
- It does not make latency-sensitive polling safe merely by hiding failed polls as epsilon steps; such sites require NB-CF, isolation, direct witness construction, or Appendix L snooping.

2. Architecture at a glance



2.1 The five interfaces between appendices

Interface	What crosses it
G -> I	G supplies the operational source model, observable labels, issue/commit rules, R.RegSeq, the Policy-L/Policy-S interpretation, and the E3 implementation discipline used by local replay.
I -> J	I supplies process-local replay/preparation facts packaged as J.PCert. These facts intentionally do not claim global channel enablement or one common source execution.
L -> I/J	L supplies a witness selector proving WC when epsilon-modeled choices can affect later observable control, values, request attribution, or frontier identity.
J -> K	J supplies a reachable source witness, phase-correct reachability, and one equal KObs record at matched normalized cutpoints. K does not compare arbitrary internal states.
Policy-S -> K	SDet, K.CompleteS, K.RespCov, and K.RespClean describe one selected deterministic conservative source run. Proposed K.2b/K.2c use it to discharge A5-L on the supported fragment; this reduction remains pending mechanization.

2.2 Generic theorem machinery versus design-specific evidence

The framework deliberately separates proof algorithms that should be mechanized once from evidence that must be supplied or checked for each design, theorem instance, or RTL build.

Category	Examples
Reusable formal machinery	Definitions of trace compatibility, local-to-global construction, dependency ranking, finite-ideal induction, phase normalization, KObs reconstruction, and WFG extensionality. The Policy-S serial-dominance reduction is proposed reusable machinery whose proof and certificate interfaces still require mechanization.
Per-design source evidence	Source well-formedness, unique signal anchors, R.RegSeq classification, WC/NB-CF/snooping coverage, shared-memory lowering, barrier well-formedness, and value-determinism conditions.
Per-RTL/tool evidence	E3 scheduling conformance, E4 channel realization, exact capacities and boundaries, B-faithfulness, request attribution, reset behavior, and local process/channel/atomic certificates.
Per-liveness-instance evidence	The fixed capacities, elaboration, stimulus/environment, witness, simulator semantics, Policy-S run identity, response-monitor coverage/bounds, cutpoint coverage, and the selected A5-L discharge route.

3. Core vocabulary and proof boundaries

PURPOSE

Define the small set of objects that repeatedly connect G, I, J, K, and L.

Engineer takeaway: Most confusion disappears once three distinctions are maintained: observable events versus epsilon micro-steps; committed history versus current residual offers; and safety correspondence versus liveness assumptions.

Term	Engineer-oriented meaning
Sigma (Σ)	The selected alphabet of observable events: committed Push/Pop transfers, synchronization events, and selected signal Read/Write events. The proof boundary determines which internal or DUT-boundary signals/channels are included.
Epsilon (ϵ)	Internal behavior not directly included in Σ : pipeline advances, internal FSM transitions, failed non-blocking polls, arbitration bookkeeping, and similar micro-steps, subject to WC and K-epsilon restrictions.
Observable unit	A singleton Σ event or an explicitly atomic same-cycle group such as a rendezvous Push/Pop, paired synchronization transaction, R2C fixed-point unit, signal-anchor group, or RESET unit.
Commit	The clock cycle in which the selected boundary observer sees the operation complete; for message passing, both valid and ready are true and the payload is taken in that cycle.
Issue	The cycle attributed to a dynamic source message-operation instance when its endpoint-owned request begins. Issue attribution is operational evidence; it is not generally reconstructible from committed labels alone.
ReturnedAtCycle	The cycle boundary at which a message-call result becomes process-visible. For a successful blocking or non-blocking call it is the transfer-commit cycle; for a failed PushNB/PopNB it is the cycle in which the failure status returns even though no message commits. It is an operational timestamp, not a KObs field.
Sys_ref	The prescribed source reference transition system: raw rendezvous Sys, capacity-annotated Sys_B, or elaborated Sys_B ^{elab} with explicit staged/bundle/join/epoch-gate boundaries.
Horizon	The selected cycle/decision horizon used by J to group construction actions and enforce registered result availability and L1/L2/L3 phase ordering.
Finite ideal	A finite down-closed set of observable units: if it contains a unit, it also contains every unit required to precede it. Appendix J constructs the source witness through a chain of these legal partial-order prefixes.
KObs	The compact cutpoint record $\langle E, w, H, O \rangle$: elaboration E, witness w, canonical replay history H, and attributed residual offers O.
Residual offer	A typed current outstanding source-level request at an explicit abstract boundary, including process, frontier item, dynamic call, direction, and reset epoch.
WFG	Wait-for graph over internal processes. An edge $P \rightarrow Q$ exists when a minimal pending request of P is disabled and Q owns the unique complementary endpoint.

CUTPOINT BOUNDARY: KObs is not full-state equivalence

KObs records enough information to reconstruct the Appendix K predicates: process replay slice, explicit channel state, pending/frontier classification, boundary requests, channel enablement, wait-for edges, and snapshot drain closure. It intentionally omits arbitrary internal RTL state and historical Issue timestamps. Therefore equal KObs is a carefully designed quotient, not a claim that Sys_ref and RTL_B are in identical states.

3.1 Safety direction: why Post -> Ref is the normal signoff argument

For ordinary safety signoff, the useful deductive direction is: every covered reachable RTL prefix has a matching reachable Sys_ref prefix with the same canonical observable history. Therefore, if all relevant Sys_ref histories satisfy a prefix-closed safety property, the RTL histories do as well. This is Corollary J.1-Safe.

The reverse direction is intentionally restricted. A particular source prefix is guaranteed to have an RTL realization only when it is already equipped with an RTL-consistency annotation package, or when Appendix L selects the source witness to align with the actual RTL choices. The document does not claim unrestricted equality of the source and RTL behavior sets.

4. Appendix G - the observable scheduling contract

PURPOSE

Define what behaviors are observable, which source schedules are legal, and what the post-HLS implementation must preserve.

Engineer takeaway: Appendix G is the semantic contract. It does not by itself construct a matching source execution; Appendices I and J do that work.

4.1 Source-side rules R1-R5

Rule	Role in the architecture
R1 - synchronization order	Synchronization calls of each process remain in dynamic source order and commit in strictly increasing cycles.
R2 - signal anchoring	For a sequential process, a signal Read is logically anchored to the closest preceding synchronization; a signal Write is anchored to the closest succeeding synchronization.
R2C - combinational fixed point	Combinational signal observations are emitted at a deterministic unique epsilon-fixed point for the explicitly modeled combinational component.
R3 - sync isolation	A synchronization boundary separates the message operations in the immediately preceding and succeeding source intervals.
R4 - safe issue order	Dynamic message operations preserve source issue order, with a prefix-closed reading: a later operation cannot issue unless every required earlier operation has issued.
R5 - channel capacity semantics	The source reference uses rendezvous or finite-capacity channel semantics at the selected explicit abstract boundaries; elaboration may introduce proof-internal channels but does not reorder ordinary source calls.

DEFINITION: R.RegSeq - registered sequential-interface semantics

For a sequential process, request and Push-data outputs in cycle t depend only on registered state and operands fixed before L2 settle. If $\text{ReturnedAtCycle}(q, t)$ holds - for a success or failed non-blocking return - any dependent evaluation, control, endpoint/address, payload, or issue has a later Horizon. ReturnedAtCycle is defined normatively in Appendix C; R.RegSeq states the dependent-use rule.

- **Why it matters.** It eliminates same-Horizon decision-to-dependent-request edges that would be inconsistent with clocked hardware.
- **Where it is used.** Appendix I local preparation, J.FoundationRank/J.DepRank, J.RegPhaseNF, and the Appendix K serial-reduction assumptions.
- **DV implication.** Checked source/tool evidence must establish the registered-cycle rule. Zero-time dependent NB cascades must be identified and then rejected, excluded, isolated, remodeled atomically, or aligned through Appendix L; silent scope exclusion is forbidden.

4.2 Operational issue policies

Appendix G treats the source model as an operational transition system with explicit local process state, explicit channel state, dynamic operation identities, pending requests, and epsilon micro-state. Two source issue policies are important:

Policy	Meaning and use
Policy L - liberal	The permissive issue rule used by the safety witness and by A5-L. It allows reached, outcome-independent operations to issue subject to source order, R.RegSeq, channel semantics, and the selected witness. Definition R.LAdm supplies the exact legal-path and cycle-phase convention; basic Policy L is not itself the continuation-level drain qualification.
R.LAdm - admissible Policy L	The Appendix K admissibility qualification on Policy-L paths, not a different issue policy. It fixes L0-L4 and additionally requires nonwithdrawal, K-epsilon, and activated R.16/K.Drain after a settled fully disabled frontier. For pipelines, Appendix B automatic flush is the primary discharge.
Policy S - serial blocking	A conservative process-facing policy in which a later blocking operation in the same source interval does not issue until required earlier blocking operations committed in a strictly earlier cycle. It is the selected deadlock-stress execution in the proposed Appendix K serial-reduction route.

DEFINITION: R.LAdm - L0/L1/L2/L3/L4 phase discipline

Each cycle has a process-owned pre-settle window (L1), one epsilon-settle snapshot with requests frozen (L2), boundary decisions and commits (L3), and registered result availability for the next cycle (L4). For Appendix K, a Policy-L path is admissible only if it follows these phases and, from every L2 snapshot exposing a fully disabled minimal frontier, satisfies request nonwithdrawal, K-epsilon, and R.16/K.Drain: no new source-later work is launched and only finite, non-replenishing drain may continue. This is not an extra pipeline rule beyond Appendix B; automatic flush is the normal pipeline proof of this obligation, while ordinary non-overlapped control needs the equivalent no-new-later-work/drain argument.

4.3 Post-HLS compatibility obligations E1-E5

Obligation	What the RTL/tool flow must establish
E1	Preserve each process's synchronization-event order.
E2	Preserve anchored signal values/visibility for sequential processes and fixed-point signal units for combinational processes.
E3	Preserve safe message Issue order and attribution, including same-interface order, distinct-interface no-reverse order, and the prefix-closed Issue condition.
E4	Realize legal rendezvous or finite-capacity FIFO behavior at each selected explicit abstract channel boundary, including payload/order/occupancy behavior.
E5	Do not move a message operation across its immediate synchronization boundary: predecessor operations issue no later than the sync; successor operations issue strictly later.

BOUNDARY CONDITION: B-faithfulness and Sys_B^elab

E4 only has meaning at a chosen abstract boundary. The flow must show that every transfer crosses that boundary, occupancy and enablement match the declared capacity B(c), internal movement is epsilon-opaque, and no hidden grant, throttle, backpressure, join, or epoch-gate condition changes availability. If ordinary Sys_B cannot express the realization, Sys_B^elab must expose the additional boundaries explicitly.

4.4 Trace compatibility is observational, not temporal equivalence

The relation approximately-equal ($\approx$) matches dynamic occurrences and values at the selected alphabet while preserving the explicit constraints above. Independent observations may occur in different cycles or be grouped differently in the two executions. The relation is directed and annotation-bearing; it is not a symmetric mathematical equivalence over bare label sequences.

Appendix G also separates the explanatory causal-dependency graph from the well-founded order used by Appendix J. Causal dependency captures information flow and may contain legal same-cycle atomic cycles. Appendix J therefore constructs a narrower required-order relation and a separate generated dependency graph for the proof induction.

READING RULE: What to take from Appendix G

For architects: G defines what implementation freedom remains invisible to a latency-insensitive environment. For DV: G identifies the observable checker boundary and the conformance obligations. For formal implementers: G defines the transition-system vocabulary that I, J, and K consume.

5. Appendix I - per-process replay and preparation

PURPOSE

Show that each process can locally replay the selected RTL observable prefix and prepare for its next participation using a finite source-owned script.

Engineer takeaway: Appendix I is intentionally local. It identifies the right dynamic operation and reaches its request/decision point, but it does not assert that the complementary endpoint is present or that all process scripts can be merged globally.

5.1 Inputs and preconditions

Input	Why I needs it
Fixed initial state and stimulus	The process is replayed under the same global Sigma-causal environment behavior, not an arbitrary P-local stimulus.
WC witness	Every epsilon-modeled choice that can affect later observable control, values, identities, requests, or frontiers is fixed consistently.
I.A0 / E3	The RTL message-issue discipline is supplied as an implementation assumption, including prefix-closed issue attribution.
R.RegSeq	Dependent next-cycle requests cannot be created from same-cycle returned results.
B4 / finite pipeline depth	Internal micro-progress has a finite epsilon bound when the process is not externally stalled.
B2/B3 when progress is invoked	Continuously enabled actions and complementary channel discharge are eventually selected; these are environment/scheduler assumptions, not consequences of source code.
Source well-formedness	Signal anchors, dynamic operation identities, direct-input contracts, shared-memory lowering, and reset behavior are well formed.

5.2 The replay slice: I.Obs, I.Obs-Congruence, I.DR, and I.DR-prime

The key simplification is to replay only the source-state facts that can affect later observable behavior or Appendix K's request/frontier predicates. I.Obs includes control location, future-relevant variables, dynamic operation identities, pending/frontier/request attribution, relevant witness state, and anchored values. It ignores diagnostic state and epsilon-only timing state that cannot influence the proof interface.

LEMMA: I.Obs-Congruence

If two epsilon-quiescent source states have the same Sigma-relevant slice and take the same next observable label under the same witness, their successor Sigma-relevant slices remain equal. This is the one-step closure property that makes deterministic replay possible.

LEMMA: I.DR and I.DR-prime - deterministic replay

I.DR establishes equality of the Sigma-relevant source slice after the same observable prefix at normalized cutpoints. I.DR-prime gives the corresponding immediate post-unit state, which is important because Appendix J constructs the next Horizon without automatically normalizing after every unit.

5.3 Deferred non-blocking decisions

A process-local proof cannot decide a non-blocking poll from process state alone because success or failure depends on the common incident-channel snapshot and possible complementary requests. Stopping the local replay at every poll would be too restrictive: a registered process may execute independent same-Horizon work that does not depend on the poll result.

DEFINITION: I.DeferredNBHole

A deferred hole records the dynamic non-blocking call, explicit boundary, snapshot key, Horizon, result destinations, reset epoch, witness-selected expected outcome, and NoDependentUse evidence. The process-local preparation script can carry the unresolved hole through independent work while forbidding any dependent use of its result.

5.4 The local preparation theorem

LEMMA: I.3 - finite process-local preparation script

Given the replay state, exact dynamic operation identity, R.RegSeq, WC, E3/I.A0, source well-formedness, and typed footprints, I.3 constructs a finite process-owned epsilon/request path to the selected operation, its persistent issued request, or a carried deferred non-blocking decision hole. The script does not commit an unmatched observable unit and does not assume global enablement.

- **Output.** The information later packaged in J.PCert: replay state, dynamic identity, finite per-Horizon actions, request/pending attribution, stable payload/value facts, footprints, and deferred-hole records.
- **Non-result.** No complementary endpoint, common channel snapshot, globally legal merge, or reachable system extension is proved here.
- **Why finite.** The source-control segment is finite and internal progress is bounded by B4/FPD when not externally stalled.

5.5 Engineer example: independent work after a poll

Suppose a process performs a PopNB, updates an independent diagnostic counter, prepares a request on another endpoint whose control and payload do not depend on the PopNB result, and only later branches on the poll status. Appendix I may carry the poll as a DeferredNBHole through the independent counter/request work. Appendix J later freezes the complete channel request snapshot at L2, decides the poll once globally, and prevents the branch from occurring until the result is available under R.RegSeq.

ARCHITECTURAL SEAM: Why Appendix I cannot be the whole safety proof

Local scripts may each be valid when viewed against read-only summaries, yet still conflict through shared channel state, atomic rendezvous pairing, a common non-blocking snapshot, reset scope, or overlapping field accesses. Appendix J proves that the scripts coexist and derives a legal global order; it does not merely conjoin them.

6. Appendix J - local-to-global safety construction

PURPOSE

Construct one reachable Sys_ref execution from local process/channel/atomic certificates and prove observable compatibility with a selected RTL prefix.

Engineer takeaway: Appendix J is the main safety theorem. Its defining achievement is that global witness realizability is derived from local evidence rather than accepted as an unexplained system-level premise.

6.1 Theorem J.1: the top-level safety result

THEOREM: J.1 - execution-level / trace-set compositional compatibility

For every covered reachable RTL_B prefix, the normal compositional route uses Definition J.LocalGWR and Theorem J.GWR-Comp to construct a reachable Sys_ref prefix and a global witness package whose annotated projections satisfy E1-E5. A directly validated J.GWR package is an alternate route. The reverse Ref -> Post clause applies only to source prefixes already equipped with an RTL-consistent annotation/witness package.

Three details are important for engineering use:

- **Finite reachability, not only fair executions.** The Post -> Ref construction applies to finite reachable RTL prefixes; fair extension is a separate premise when an infinite execution is needed.
- **Required-prefix induction.** The proof orders observable units by the constraints that must be preserved, rather than by raw RTL cycle buckets or full source order across independent endpoints.
- **Annotated carrier.** Compatibility includes dynamic identities, witness choices, Issue/Commit attribution, request facts, and channel histories. It is stronger than matching an unannotated list of labels, but weaker than full-state equivalence.

6.2 The local certificate package

Component	What it certifies
J.PCert	Per-process replay and finite preparation evidence produced from Appendix I.
J.CCert	Per-explicit-channel boundary identity, capacity, E4 history, occupancy/head, payload, request snapshot, enablement, reset, and B-faithfulness evidence.
J.ACcert	Agreement for explicitly atomic units: rendezvous pairs, paired synchronization, E2/R2C groups, RESET, and other named multi-participant actions.
J.EnvCert / J.ResetCert	Environment and reset-unit identity, scope, action, and locality evidence.
Typed footprints	Field-sensitive ReadSet/WriteSet evidence for every construction action, including component-local elaboration actions.
LocalAgree	Equality, uniqueness, ownership, coverage, and duplicate-field agreement across local records.
Generated dependencies	Locally justified Dep_J edges and enough evidence to prove that omitted pairs commute; the package does not supply a convenient global rank.

DEFINITION: J.LocalGWR

The complete local package for one RTL prefix: PCert for every process, CCert for every explicit boundary, ACert for every atomic unit, applicable environment/reset records, complete action/deferred-hole inventory, field-sensitive footprints, R.RegSeq and source conformance, generated dependencies, LocalAgree, and coverage. It must not assume that these records already form a realizable global replay.

6.3 How J derives a global construction order

J converts the certificate package into a finite graph of primitive construction actions. The graph includes process epsilon/control actions, request/attempt assertions, settled-snapshot decisions, complete commits, and reset/environment actions. Atomic groups are contracted before ordering.

Definition / lemma	Role
J.DepJ	Generated direct dependency edges from process control/data/Issue/ReturnedAtCycle rules, request-before-snapshot rules, channel/FIFO state, atomicity, reset/environment order, and explicit component dependencies.
J.DepSound	Every generated edge is a real operational prerequisite.
J.DepComplete	If two non-atomic actions are left unordered, they either commute by field-sensitive framing or are the explicitly proved same-cycle buffered Push/Pop dual update.
J.FoundationRank	A graph-independent lexicographic semantic rank based on reset epoch, Horizon, semantic stage, local ordinal, and component tie data.
J.DepRank	Every generated edge strictly advances the foundation rank.
J.DepAcyclic	A directed cycle would strictly increase the well-founded rank back to its start, which is impossible.
J.CanonicalRank	A derived deterministic totalization of the acyclic prerequisite relation. Stable tie keys order only commuting actions.

LOAD-BEARING RESULT: J.DepComplete + J.DepRank

The flow cannot simply provide a global rank that is assumed correct. It must generate every non-commuting prerequisite from local rules, prove that unordered actions really commute, and prove each edge advances an independent semantic foundation rank. This is what makes the construction auditable and prevents a hidden global replay assumption.

6.4 Registered phase normalization

Within each Horizon, the proof must respect actual clocked interface semantics. It cannot decide a non-blocking result and then create a dependent request in the same cycle, and it cannot evaluate different polls against inconsistent partial channel snapshots.

LEMMA: J.RegPhaseNF - registered phase normal form

Every finite same-Horizon construction fragment can be permuted, without changing its canonical observable history, into: L1 process evaluation and request/attempt writes; one L2 settled common snapshot; and L3 non-blocking decisions plus complete boundary commits. Registered results become available only in L4/next cycle.

LEMMA: J.NBDecision

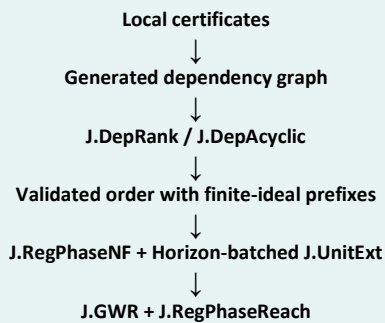
Once all L1 predecessors have executed and J.CCcert supplies the common settled snapshot, each deferred non-blocking call has exactly one legal source result. A failed poll cannot coexist with a matching enabled rendezvous request, and a successful poll cannot be invented in a full/empty state that forbids it.

6.5 From one Horizon to the complete source witness

Lemma / theorem	Function in the construction
J.Frame	A certified transition changes only its WriteSet; disjoint state and guards are preserved.
J.LocalLift	Embeds one Appendix I process-local script into a reachable global state when owned/shared locations match its certified summaries.
J.PrepareCommute	Moves independent preparation actions into the common L1 window while respecting snapshots and true dependencies.
J.UnitEnable	After L1/L2, the requests, payloads, channel state, atomic participants, and holes make each selected L3 unit genuinely enabled.
J.UnitExt	Executes one complete Horizon bucket, updates replay/channel/atomic state, resolves holes, and preserves the construction invariant.
J.GWR-Comp	Outer induction over Horizon buckets constructs the complete reachable Sys_ref witness chain and the stable J.GWR interface.

THEOREM: J.GWR-Comp - local certificates construct global witness realizability

From J.LocalGWR, the theorem validates the dependency graph and construction order, phase-normalizes each Horizon, applies Horizon-batched J.UnitExt, and produces J.GWR and J.RegPhaseReach for one reachable Sys_ref witness.



- **Non-circularity.** LocalGWR supplies no global order, no phase-normal replay, and no global extension premise.
- **Stable interface.** J.GWR can also be supplied directly by a translation validator or symbolic replay tool, but that direct route must independently validate the same reachability, ordering, and ideal-prefix interface.
- **Reset handling.** RESET is an atomic observable unit; RW1-RW5 and Lemma J.RS splice a new matched reset epoch while framing unaffected components.

Finite-ideal invariant. Every flattened prefix of the validated order is down-closed: if it contains a unit, it contains all required predecessors. J.UnitExt therefore grows the witness from one legal partial-order prefix to the next. Tie ordering among independent units is bookkeeping; required predecessors are not skipped and atomic units are never split.

6.6 Proposition J.0 is deliberately separate

PROPOSITION: J.0 - pairwise composition lemma

Given an already selected compatible source/RTL trace pair, compatible per-process annotated projections, and per-channel E4 well-formedness, J.0 lifts E1-E5 to the system level. It does not construct the source trace, prove B-faithfulness, or establish J.GWR. Keeping it separate prevents circular reasoning.

6.7 Safety transfer for the verification flow

DEFINITION: J.TraceCertCov + COROLLARY: J.1-Safe

J.TraceCertCov_I(D) is the named Post $\rightarrow$ Ref safety-certificate coverage premise: every RTL prefix in the selected domain D must carry either accepted J.LocalGWR/J.GWR-Comp evidence or a directly validated J.GWR package, plus the remaining Theorem J.1 premises, all bound to the same theorem instance, stimulus, observation boundary, and prefix. Under that coverage, any prefix-closed J.HCanon-invariant safety predicate transfers from all reachable Sys_ref prefixes to every covered RTL prefix. One complete pre-HLS run is sufficient only when J.RefProjCover proves that its canonical observable prefixes cover all relevant reachable source observations. J.TraceCertCov is safety-side coverage only and does not imply K.CertCov.

6.8 The J-to-K cutpoint bridge

Trace matching alone is insufficient for deadlock reasoning. Two executions can have the same committed history while one endpoint has not reached a blocking call and the other has an attributed persistent request. J therefore augments the canonical replay history with the terminal residual-offer set.

Bridge element	Purpose
J.HCanon	Proves that the selected source and RTL prefixes have the same canonical replay history H modulo the permitted epsilon/history quotient.
J.RegPhaseReach	Records that the exact source witness was constructed with R.RegSeq and the Horizon L1/L2/L3 phase discipline.
J.OfferReach	Shows that $\langle E, w, H, O \rangle$ is actually realized by a reachable normalized Sys_ref cutpoint, not merely a well-typed tuple.
J.OfferConf	Bidirectionally and uniquely matches every residual offer in O to a relevant asserted RTL request and every relevant RTL request back to exactly one offer.
J.KObsConf	Binds one exact source/RTL/cutpoint package and establishes equality of the typed KObs record.
Corollary J.1	Exports equal KObs and therefore equal reconstructed frontier, request, channel, enablement, WFG, and snapshot drain predicates to Appendix K.

AUDIT CRITICAL: J.OfferConf request-to-offer completeness

Every relevant asserted RTL boundary request must be attributed to exactly one source residual offer at the selected explicit boundary. Omitting a hidden asserted request, duplicating attribution, or using an unrelated source cutpoint invalidates the KObs bridge. This is the implementation abstraction boundary on which Appendix K relies.

7. Appendix K - conditional deadlock preservation

PURPOSE

Preserve absence of reachable drain-closed closed internal message-passing wait cycles from the selected source-reference instance to covered RTL cutpoints.

Engineer takeaway: K is not a generic progress theorem. It proves a specific reachable-cutpoint property and is conditional on A5-L plus fairness, drain, environment, value, lowering, and coverage obligations.

7.1 The deadlock predicate

Appendix K evaluates epsilon-quiescent cutpoints at explicit point-to-point channel boundaries. The WFG contains only internal message-passing dependencies; environment-facing channels and SyncChannel barriers are excluded from the graph and handled through separate scope/assumption rules.

Object	Meaning
Front_P	The minimal pending blocking message-operation items of process P whose endpoint requests are currently asserted. Merely being the next source call is not enough; the request must have issued.
Blocked process	Front_P is non-empty and every frontier item is ChannelDisabled at the selected explicit boundary.
WFG edge P -> Q	A disabled frontier request of P waits on the unique complementary endpoint owner Q.
DrainClosedNow(D)	No already-offered non-frontier request owned by D remains enabled; otherwise finite downstream drain could still change the candidate wait structure.
Definition K.Dead / K.DeadW	Dead_K is the ordinary reachable drain-closed closed internal wait-cycle predicate. Dead_K^W is the waiver-parameterized form, excluding declared KObs-closed cutpoint classes. Optional Dead_KS remains diagnostic only.

SCOPE: Reachable closed internal wait-cycle cutpoint

The operative predicate is stronger than “the system is currently slow” and different from a global permanent-deadlock definition. It identifies a reachable internally closed wait cycle at a drain-closed snapshot. Escape through a nondefault timed/reactive environment co-frontier is a separate concern controlled by K.EnvMF and explicit waivers.

7.2 Reconstructing the wait state from J.KObs

Result	Function
Lemma K.0	Reconstructs the operational minimal pending blocking frontier from KObs. It distinguishes a reached-but-unissued candidate from an asserted persistent request.
Lemma K.1a	Characterizes disabled Push/Pop frontiers for buffered and rendezvous channels at the chosen explicit boundaries.
Lemma K.2	WFG extensionality: equal KObs implies exactly equal reconstructed wait-for graphs.
Lemma K.2a	Snapshot drain-closure extensionality: equal KObs implies equal DrainClosedNow for any process set.

Result	Function
R.21 reconstruction lemmas	Factor process replay, channel state, offers, frontier, enablement, WFG, drain closure, and the deadlock predicate into pure functions of KObs.

7.3 One certified cutpoint versus universal coverage

DEFINITION: K.CertifiedCutpoint

A K-ready package for one finite reachable RTL prefix τ_{post} , one explicitly recorded selected finite normalization witness ν_{post} , and the K-relevant epsilon-quiescent endpoint ρ_{post} of that same witness. It binds the theorem instance, records ν_{post} , and contains the J.1/KObs correspondence, exact source witness, J.RegPhaseReach, K.DrainReach, point-to-point boundary facts, waiver closure, and the remaining K-facing assumptions for that one prefix/cutpoint pair. It does not imply K.CertCov.

THEOREM: K.1A - certified-cutpoint preservation from A5-L

Assume one certified RTL cutpoint satisfies Dead_K^W . Equal KObs transfers the same frontier/WFG/drain predicate to the exact reachable Sys_ref witness. J.RegPhaseReach plus K.DrainReach prove that witness is R.LAdm-admissible. This contradicts A5-L. Therefore that certified RTL prefix/normalization/cutpoint package cannot contain the non-waived internal wait cycle.

COROLLARY: K.1A-Total - universal result under K.CertCov

K.1A is pointwise. A universal claim requires K.CertCov over every reachable triple $(\tau_{\text{post}}, \nu_{\text{post}}, \rho_{\text{post}})$: the finite RTL prefix, the selected finite normalization witness, and its K-relevant epsilon-quiescent endpoint. The normalization choice is proof-relevant, so the theorem instance must fix a selection function or record the equivalent witness in each package. One successful CF-0/K.CertifiedCutpoint check does not establish K.CertCov.

7.4 The central source-liveness premise A5-L

PREMISE: A5-L - scheduler-robust liberal source liveness

No finite Policy-L prefix admissible under R.LAdm for the selected Sys_ref instance reaches an epsilon-quiescent non-waived Dead_K cutpoint. Because Policy-S-shaped serial prefixes are included among admissible Policy-L prefixes, a design with a reachable serial wait cycle makes A5-L false; another proof technique cannot certify the unchanged instance.

A5-L is intentionally global. Internal wait cycles depend on interactions among processes, capacities, environment behavior, and the selected issue policy. Unlike the J safety construction, it cannot generally be reduced to independent per-process liveness theorems.

7.5 The primary Policy-S discharge route

The proposed practical route avoids exploring all liberal schedules directly. It fixes one fully specified deterministic conservative source simulator and aims to prove that any admissible liberal internal deadlock would cause a finite, covered process-facing response failure on that same selected serial execution.

Element	Role
K.Low / Sys_K	Normalize synchronization waits and witness-selected non-NB-CF dependencies into explicit blocking guard/epoch-gate channels so the serial argument operates on literal blocking frontiers.
SDet	Fix every history-affecting choice: model, capacities, elaboration, reset, stimulus/environment, witness, arbitration, concrete simulator scheduling, Policy-S issue semantics, and epsilon normalization. Select one run artifact $\rho_{S^{\wedge}det}$.
K.EnvMF	For multi-frontier components, require StandardEnv/B1-available plus the default ready output environment; timed/reactive or otherwise nondefault environments are restricted to a certified single internal frontier.
K.RespCov	Default-total coverage of every static internal process-facing blocking site and relevant dynamic class, with a finite bound or justified disposition.
K.RespClean	No finite prefix of the complete selected run witnesses a non-waived timeout, maximal-finite pending frontier, cancellation/reset failure, or end-of-test pending failure.
K.CompleteS	The selected run is maximal and fair. A terminating run reaches a declared terminal state; a nonterminating run requires invariant/model-checking/lasso or equivalent completeness evidence.
A5-S / CF-S	The package K.CompleteS + K.RespCov + K.RespClean for the SDet-selected run, bound to one certificate header/trust boundary.

PROPOSED LEMMA: K.2b - deterministic serial-source dominance with response-failure extraction

The proposed lemma states that, under the blocking/lowering, value-determinism, environment, fairness, drain, identity, and coverage premises, any R.LAdm-admissible Policy-L internal deadlock forces the same SDet-selected Policy-S execution to produce a finite non-waived K.RespFail witness. K.2b.Q Step 1 claims prefix completion only for the process-facing blocking Push/Pop operations required by the Sys_K/K.BF serial prefix. Failed non-blocking polls are handled through their WC-selected return outcomes and are not claimed to have committed. The lemma does not claim that the two runs have the same trace or deadlock component; the response-failure target remains the central mechanization-sensitive dominance claim.

PROPOSED COROLLARY: K.2c - the selected deterministic Policy-S execution suffices

If the complete selected Policy-S run has total response coverage and is response-clean, proposed K.2b rules out any liberal deadlock counterexample; therefore A5-S implies A5-L on the supported fragment. SDet removes quantification over alternate Policy-S schedulers, so no confluence theorem is required. This implication remains proposed until the serial-dominance and certificate obligations are mechanized.

PROOF STATUS: Policy-S serial reduction is plausible, but not yet certified

The approach is plausible because Policy S is a conservative stress schedule; K.2b seeks only a process-facing response failure rather than exact trace/deadlock replay; SDet fixes one concrete run and avoids a confluence problem; and K.Resp monitors the blocked process instead of requiring an exhaustive classification of the remote owner.

What Lean must still establish. Identity-aware first-violation/frozen-component dominance, buffered and rendezvous request reachability, multi-frontier joint freeze and monitor composition, fair complete frozen extension (K-O7), default-total K.RespCov, and finite K.RespFail extraction. Mechanization may validate the current fragment or reveal that an additional side condition or narrower scope is needed.

PRACTICAL WARNING: A finite clean prefix is not enough

For a terminating design, the run must reach a declared maximal terminal state with all covered monitors resolved. For a nonterminating design, a finite simulation trace is only evidence; a complete invariant, sound model-checking result, recurrence/lasso proof, or equivalent argument must establish that response failures never occur.

7.6 Alternate direct discharges of A5-L

Route	When it can discharge A5-L
K.Str-1 - acyclic dependency graph	If the full static process dependency graph is acyclic, no WFG cycle can occur.
K.Str-2 - ranked dependency	A well-founded rank strictly decreases along every realizable WFG edge, excluding cycles.
K.Str-3 - credit/token cycle breaking	Every static dependency cycle contains an edge proved unrealizable at reachable cutpoints, often by occupancy, initial-token, or credit invariants.
CF-3 / direct invariant	An inductive abstraction proves the prohibited Policy-L cutpoint unreachable.
CF-4 / exploration	Sound exhaustive or bounded model checking over the selected transition system.
CF-5 / tool certificate	A tool-emitted certificate using knowledge of the scheduled control graph, storage, arbitration, joins, and automatic flush.

These are alternate proofs of exactly the same A5-L property. They cannot rescue a design for which an admissible serial-shaped Policy-L prefix already reaches Dead_K.

7.7 Standing assumptions A1-A11 in engineer terms

Assumption	Plain-language role
A1	The source and RTL satisfy the previously defined R-rules and B-rules at the selected boundaries.
A2	Finite internal epsilon depth / finite stutter when not externally stalled.
A3	Weak fairness: a continuously enabled action is eventually selected.
A4	Channel progress: persistent complementary requests eventually discharge full/empty/rendezvous blocking.
A5	For the user-facing Policy-S route, a complete bounded-response-clean selected serial run; K.1A itself consumes A5-L directly.
A6	Point-to-point channels with unique producer and consumer at every evaluated boundary.
A7	B1 RTL determinism/witness selection plus the exact equal-KObs source/RTL correspondence and R.LAdm qualification.
A8	Barrier participation is well formed; barrier mismatches are outside the internal message-passing deadlock theorem.
A9	At most one Push and one Pop commit per channel per cycle; multi-lane/burst resources must be modeled explicitly.
A10	Every buffered channel is logically empty after each reset boundary.
A11	K-epsilon: hidden epsilon steps do not create, remove, or redirect the WFG-relevant blocking structure.

THEOREM: K.1 - user-facing deterministic Policy-S preservation

Combine uniform K.1A premises, K.CertCov, SDet, K.CV, applicable NB-Align/K.Low, point-to-point boundaries, nonwithdrawal, K-epsilon, K.Drain, K.EnvMF, K.RespCov, and A5-S. Proposed Corollary K.2c supplies A5-L; K.1A-Total then excludes the prohibited reachable internal wait-cycle cutpoint in RTL_B.

Status. The Policy-S reduction form of K.1 remains a proposed result until K.2b/K.2c, the fair frozen-extension lemma, total response coverage, failure extraction, and the associated certificate interfaces are mechanized. K.1A remains the direct theorem interface when A5-L is discharged by another valid route.

7.8 What Appendix K does not cover

- Barrier mismatches or conditional SyncChannel participation (handled by A8 and separate protocol verification).
- Waiting indefinitely for an external environment input, output readiness, termination, EOF, cancellation, or reset behavior not represented by the internal WFG predicate.
- Arbitrary starvation or throughput guarantees unrelated to the defined blocking frontier and response-monitor schema.
- Cycle-accurate RTL response bounds; K.Resp evidence is a source-side discharge of A5-L, not an RTL timing contract.
- Designs whose correctness requires favorable eager/same-cycle issue when a serial-shaped admissible prefix deadlocks.

8. Appendix L - witness selection and snooping

PURPOSE

Conditionally realize a WC-compatible source witness for latency-sensitive epsilon choices while leaving the unique free-running RTL behavior untouched.

Engineer takeaway: Snooping selects one admissible source witness only when the required WC/L.Dir witness exists. It must not feed pre-HLS readiness back into RTL, relabel payloads, drop/reorder observations, or treat an unrealizable witness as a silent stall.

8.1 Why snooping is needed

A non-blocking arbiter, interrupt controller, or retry/timeout block may make a later observable choice based on which request arrived first or how many polls failed. HLS changes internal latency, so the raw pre-HLS and post-HLS simulations can select different epsilon outcomes even though both satisfy the broad scheduling contract. When those outcomes affect later Sigma-visible behavior, Appendix I needs a specific WC-compatible witness.

If NB-CF holds - failed non-blocking outcomes do not affect later observable control, values, frontier, or request attribution - the witness is trivial and snooping is unnecessary. Cadence-sensitive polling is different: it is a localized latency-sensitive protocol and must be isolated or explicitly snoop-aligned. Appendix L is conditional: if no admissible source execution can consume the RTL event sequence in ordinal order with matching kind and payload, WC/L.Dir is not discharged for that comparison.

DEFINITION: L.1 - witness selector / snooping

At each relevant epsilon-modeled source choice, the selector returns the outcome chosen by the matched RTL prefix using only causally available history. It cannot depend on future RTL events or invent a retry/timeout outcome inconsistent with the isolated protocol.

LEMMA: L.2 - conditional snooping discharge of WC

If the snooped set covers every epsilon choice that can affect later observable behavior, the wrapper changes only epsilon-choice selection, selected outcomes are causal and RTL-consistent, successful and failed non-blocking calls retain the required Sigma/epsilon interpretation, cadence-sensitive blocks are isolated, and the selected ordinal event stream is realizable, then the selected source execution satisfies WC for Appendices I and J. This is a conditional witness-construction theorem, not a guarantee that every RTL event stream has a matching source witness at the chosen boundary.

8.2 One-directional architecture

CONDITION: L.Dir - one-directional alignment / RTL-side non-interference

The RTL_B execution is free-running under the fixed stimulus. The pre-HLS snoop sequence must consume the recorded RTL snoop sequence in ordinal order, with matching kind and payload, and no pre-HLS backpressure/readiness may stall or alter RTL. Benign cycle skew is permitted. Order, kind, or payload divergence means that the required WC witness does not exist at the chosen boundary; the run has no theorem-backed guarantee and the harness must abort and triage the failure rather than silently gating or hanging.

8.3 Wrapper realization requirements W1-W5

Requirement	Engineering meaning
W1 - recorded ordinal observation	Capture each RTL snooped event by dynamic ordinal and hold it until the pre-HLS side consumes the same ordinal. Do not rely on overlapping live assertion windows.
W2 - gate only commit enable	Gate only the mirror signal that enables the pre-HLS commit: observed valid for a pre-HLS Pop or observed ready for a pre-HLS Push. Never modify the source endpoint-owned request.
W3 - registered sampling	Register the RTL observation before it enters the source-side gate to avoid delta-cycle races; the extra source latency is part of witness selection.
W4 - compare and watchdog	At each source snoop commit, compare ordinal, kind, and payload against the recorded RTL event. Never substitute the RTL payload into the source, because that would hide value divergence. Detect missing, extra, reordered, or never-consumed observations. A watchdog must convert non-existence or failure to realize the witness into a reported result; if the watchdog bound is not proved sufficient, a bound-exceeded abort is an inconclusive witness-realization failure, not by itself a proved L.Dir violation.
W5 - independent coverage proof	The wrapper mechanism is sound only for the points it records. A separate analysis must prove that all epsilon choices capable of affecting later observables are covered.

IMPLEMENTATION PITFALL: Why a live AND is unsound

ANDing the current pre-HLS and current RTL handshake levels can drop an RTL event when the RTL assertion retires before the source arrives. It can also reorder or duplicate matching under multiple outstanding events. The normative rule is an AND against a recorded per-ordinal observation retained until source consumption.

8.4 Harness soundness and shared testbenches

LEMMA: L.5 - online realization of the witness selector

A co-simulation harness realizes the abstract selector only when a matching witness exists, the harness preserves the free-running RTL trace, implements W1-W5, satisfies L.Dir, and produces the same source-side witness choices assumed by the formal comparison. The formal theorem is about the fixed RTL trace and selected source witness; harness correctness, coverage, and successful witness realization are additional implementation obligations.

A shared reactive testbench becomes problematic once one model can lag significantly, because reactions may be driven by whichever model the testbench observes. The flow should replicate the stimulus/testbench per model, or structure a shared environment around a common causal transaction stream rather than raw cycle timing.

8.5 What snooping means formally

Snooping does not use RTL to change the source specification. The source model is intentionally under-specified with respect to permitted latency and epsilon choice. When a WC/L.Dir witness exists, the RTL observation selects one admissible execution of that specification for comparison. If the event order, kind, or payload cannot be realized, the comparison is outside theorem scope and must be triaged among source-model ill-formedness, incomplete snooping coverage/harness behavior, and tool/RTL non-compliance.

8.6 Latency-robustness stress testing

Appendix L also recommends randomized stalls, varied channel latency/capacity, and adversarial weakly fair scheduling in the pre-HLS model. This is not a substitute for the formal premises, but it is a practical way to detect designs that accidentally rely on incidental ordering of causally independent events. A failure under such stress usually indicates that the source specification itself is outside the intended latency-insensitive model or that an explicit snoop/isolation boundary is missing.

9. How the appendices work together in a verification flow

PURPOSE

Translate the theorem architecture into a concrete signoff workflow.

Engineer takeaway: Safety-only use stops after the J.1/J.1-Safe package. Appendix K is added only when the claim includes the defined internal wait-cycle freedom, and it requires substantially more global evidence.

9.1 Step-by-step safety flow

Step	Deliverable
1. Fix the theorem instance	Select Sys_ref/Sys_B/Sys_B^elab, explicit boundaries, capacities, observable alphabet, reset interpretation, fixed stimulus/environment, witness provenance, and tool/RTL identities.
2. Establish G-level conformance	Check source R-rules, signal anchors, R.RegSeq, E3, E4, E5, elaboration, and B-faithfulness.
3. Discharge WC	Use NB-CF, an Appendix L snooping/L.2 construction with a realized WC/L.Dir witness, direct witness construction, or a complete mixed per-site coverage argument.
4. Produce local evidence	Generate J.PCert from Appendix I and per-channel/atomic/environment/reset certificates plus typed footprints and local dependencies.
5. Run J.GWR-Comp	Derive the dependency order and validated construction order with finite-ideal prefixes, then the registered phase-normal construction, reachable source witness, and J.GWR interface.
6. Apply Theorem J.1	Obtain Post $\rightarrow$ Ref observable compatibility for the covered reachable RTL prefix.
7. Transfer the safety property	Use J.TraceCertCov and Corollary J.1-Safe over the required RTL coverage domain; use J.RefProjCover only when claiming one selected source run covers all source observations.

9.2 Additional liveness flow

Step	Deliverable
8. Choose normalized cutpoints	Use B4/B5 and explicitly record the selected finite normalization witness nu_post from each reachable RTL prefix tau_post to its K-relevant epsilon-quiescent endpoint rho_post, together with the corresponding source normalization.
9. Build the KObs bridge	Discharge J.HCanon, J.OfferReach, J.OfferConf, and J.KObsConf for the same source/RTL/witness/cutpoint package.
10. Qualify the source witness	Use theorem-derived J.RegPhaseReach plus explicit K.DrainReach to prove K.RLAdmReach.
11. Certify the cutpoint	Assemble K.CertifiedCutpoint for the same (tau_post, nu_post, rho_post), including point-to-point boundaries, waiver closure, exact source witness, J.RegPhaseReach, K.DrainReach, and all remaining K-facing assumptions.
12. Discharge A5-L	Use the primary SDet + A5-S/CF-S + K.2c route, or a valid structural/invariant/exploration/tool route.
13. Establish coverage	Apply K.1A to one recorded prefix/normalization/cutpoint package. For K.1A-Total or the universal user-facing K.1 claim, prove K.CertCov over the complete reachable (tau_post, nu_post, rho_post) domain; a single CF-0 package is insufficient.

9.3 Recommended reading order in the scheduling-rules document

- Start with the front Abstract, Formal Guarantees at a Glance, and the unlettered Guide to the Formal Appendices; use Appendix M as the readable Formal Results Overview, then read Appendix G for the normative contract and operational policies.
- Read the Common Formal Definitions next, especially KObs, B-rules, and the side-condition discharge record.
- Read Appendix I to understand exactly what is local and why deferred non-blocking holes are needed.
- Read Appendix J as the central safety construction; focus first on J.LocalGWR, J.DepRank/J.DepAcyclic, the finite-ideal construction invariant, J.RegPhaseNF, J.UnitExt, J.GWR-Comp, Theorem J.1, J.TraceCertCov/J.1-Safe, and Corollary J.1.
- Read Appendix L before K when witness-selected non-blocking or latency-sensitive interfaces are present.
- Read Appendix K for Definition K.Dead/K.DeadW, KObs reconstruction, the explicit (tau_post, nu_post, rho_post) certified-cutpoint and coverage interfaces, A5-L, and the proposed Policy-S discharge route. Appendix K-P is the deeper proof companion and records the remaining mechanization obligations for serial reduction.

10. Assumptions and evidence matrix

PURPOSE

Summarize the named assumptions that most often determine whether a theorem instance is usable.

Engineer takeaway: The important audit question is not merely “is the assumption plausible?” but “what artifact establishes it for this exact source, RTL, capacity/elaboration, stimulus, witness, reset, and environment instance?”

Premise / evidence	Where it matters and typical discharge
B1 - RTL deterministic trace	Consumed by J/K/L selected-trace reasoning. Evidence: RTL/tool determinism proof, qualification, deterministic arbitration/environment setup, or a validated witness-selection setup.
B2 - weak fairness	Consumed by I progress embedding and K drain/progress arguments. Evidence: scheduler/arbiter fairness proof, assertions, or Appendix N pattern qualification.
B3 - channel progress	Consumed by I.2 and K. Evidence: persistent complementary endpoint requests plus E4/B2 or a per-boundary progress certificate.
B4/B5 - finite epsilon closure	Consumed by I local finite preparation and J/K normalized cutpoints. Evidence: finite pipeline/FSM depth and a unique quiescent combinational fixed point.
WC	Consumed by I and J; provenance may be NB-CF, L.2 snooping, direct witness construction, or mixed per-site coverage.
R.RegSeq	Consumed by I/J/K. Evidence: sequential interface structure, registered request/data outputs, and no same-cycle dependent request from a returned result.
J.LocalGWR	Consumed by the normal J.GWR-Comp route. Evidence: complete local process/channel/atomic/env/reset records, footprints, dependencies, agreement, and coverage.
B-faithfulness	Consumed by J/K at every explicit abstract boundary. Evidence: back-annotation, structural RTL checks, interface monitors, refinement checks, or tool certificates.
J.TraceCertCov	Consumed by J.1-Safe over a selected RTL-prefix domain D. Evidence: every prefix in D has accepted compositional or direct J.GWR evidence plus all remaining Post -> Ref premises, bound to the same theorem instance, stimulus, boundary, and prefix. It is safety coverage and does not imply K.CertCov.

Premise / evidence	Where it matters and typical discharge
K.Drain and K-epsilon	Consumed by K.RLAdmReach, K.1A, and serial dominance. For pipelines, Appendix B automatic flush is the primary R.16/K.Drain discharge; ordinary control needs an equivalent no-new-later-work and finite non-replenishing drain proof, plus no hidden WFG-changing epsilon behavior.
J.OfferReach / J.OfferConf / J.KObsConf	Consumed by Corollary J.1 and all K cutpoint reasoning. Evidence must bind the exact same source/RTL prefix, witness, history, offers, and normalized cutpoints.
K.CertCov	Consumed by K.1A-Total and the universal K.1 claim. Evidence: total K.CertifiedCutpoint coverage over every reachable (tau_post, nu_post, rho_post) triple, with the proof-relevant normalization selection fixed or explicitly recorded. One CF-0 validation is not enough.
A5-L	Consumed directly by K.1A. Evidence: Policy-S route through K.2c or a valid structural/direct/invariant/exploration/tool proof.
SDet + A5-S	Consumed by the proposed K.2b/K.2c/K.1 Policy-S route. Evidence: selected functional simulator identity, complete run proof, default-total response-monitor coverage, finite bounds, and response cleanliness. The generic reduction and certificate checks still require mechanization.
K.CV / NB-Align / K.Low	Consumed by the serial route when values, non-blocking choices, sync waits, or lowering affect future operations. Evidence: explicit causal/value determinism and local lowering correspondence.
K.EnvMF	Consumed by the serial route. Evidence: StandardEnv/B1-available for multi-frontier components, or certified single-internal-frontier structure for nondefault environments.
L.Dir and W1-W5	Consumed by the online snooping harness. Evidence: free-running RTL, ordinal recording/consumption, comparison/watchdog, noninterference, and complete choice-point coverage.

10.1 A practical certificate boundary

A useful implementation is a versioned package with a common header that binds every artifact to one theorem instance: source and RTL build IDs, selected Sys_ref/Sys_K, boundaries/capacities/elaboration, observation projection, stimulus/environment/reset model, witness provenance, simulator/scheduler/arbitration identity, and checker/schema versions. Separate payloads then carry J safety certificates, CF-0 cutpoint correspondence, and one declared A5-L discharge route.

SIGNOFF RULE: Do not mix witnesses or cutpoints

A J.OfferReach proof for one source state, a J.OfferConf proof for another RTL cycle, and a K.Drain proof for a different witness do not compose. The formal architecture repeatedly requires exact identity of the theorem instance, stimulus, elaboration, witness, H, O, source prefix, RTL prefix, and normalized cutpoints.

11. Common misunderstandings

Misreading	Correct interpretation
“Trace compatible” means cycle accurate.	No. It preserves selected labels, values, ordering, atomicity, channel semantics, and anchors. Independent events may move between cycles.
The source pass transfers by Ref -> Post.	For ordinary safety, the deductive path is Post -> Ref plus J.1-Safe: RTL adds no covered canonical safety behavior absent from Sys_ref. Ref -> Post is restricted to annotated RTL-consistent or witness-selected source prefixes.

Misreading	Correct interpretation
Per-process replay certificates already imply a system replay.	No. Shared channel state, snapshots, atomic participants, resets, and field interference can conflict. J.GWR-Comp proves the merge.
The causal-dependency graph is the J induction order.	No. Causal dependency explains information flow and may contain legal atomic cycles. J uses generated construction dependencies plus a foundation-rank proof.
KObs is a normalized execution or full state.	No. It is a typed observation quotient $\langle E, w, H, O \rangle$ sufficient for K-facing reconstruction.
No-reverse scheduling proves deadlock freedom.	No. It prevents one deadlock-introduction mechanism. Timing overlap, finite capacity, pipeline drain, environment behavior, and serial-shaped source deadlocks require Appendix K.
A clean finite Policy-S simulation proves A5-S.	Only if it is a declared maximal terminal run with all monitors resolved. A nonterminating design needs complete invariant/model-checking/lasso or equivalent evidence.
R.LAdm adds a new pipeline-drain requirement beyond Appendix B.	No. R.LAdm is the formal Appendix K admissibility interface that imports the already-required Appendix B automatic-flush behavior, plus the exact settle trigger, request persistence, K-epsilon, and finite-capacity facts, into the selected Policy-L witness.
The Policy-S reduction is already a certified theorem.	No. It is a plausible proposed reduction with explicit scope restrictions. K.2b/K.2c and the key dominance, fair-extension, total-coverage, and finite-failure extraction obligations still require Lean mechanization.
Another A5-L route can rescue a serial deadlock.	No. If an admissible Policy-S-shaped Policy-L prefix reaches Dead_K, A5-L is false for the unchanged instance.
Snooping changes the specification to match RTL.	No. It selects one behavior already admitted by the under-specified source latency/epsilon semantics.
A live AND is enough for snooping.	No. The wrapper must record per-ordinal RTL events and retain them until source consumption; otherwise events can be dropped or reordered.
Tool generation automatically discharges the proof obligations.	No. The tool or flow must emit documentation, certificates, static checks, monitors, or formal evidence for each named obligation.
K.Resp is an RTL latency guarantee.	No. It is source-side evidence used to discharge A5-L through serial dominance.
Snooping guarantees that the source will eventually match RTL.	No. L.2/L.5 are conditional on existence and realization of a WC/L.Dir witness. Event-order, kind, or payload divergence is a relational failure; an unproved watchdog timeout may be inconclusive. The failure must be reported and triaged, not hidden by gating or an indefinite wait.

12. Quick-reference index

PURPOSE	Provide a compact map from theorem/definition names to their role in the proof architecture. Engineer takeaway: Use this section as a navigation aid when reading the normative scheduling-rules document.
----------------	--

12.1 Appendix G

Name	Role
------	------

Name	Role
R.RegSeq	Registered next-cycle dependence rule.
R1-R5	Source scheduling and channel-capacity contract.
ReturnedAtCycle	Appendix C operational result-availability timestamp; successful calls use the commit cycle, while failed NB calls return status without a commit. Not a KObs field.
R.LAdm	Exact Policy-L legal-path/cycle semantics and Appendix K admissibility qualification; Appendix B automatic flush is the primary pipeline discharge of R.16/K.Drain.
Policy S	Conservative serial-blocking issue policy.
E1-E5	Post-HLS trace-compatibility obligations.
Causal dependency	Design-intent/information-flow relation, not the J induction order.

12.2 Appendix I

Name	Role
I.A0	RTL E3 message-issue discipline assumption.
I.Obs	Sigma/K-relevant source-state slice.
I.Obs-Congruence	One-step closure of the replay slice.
I.DR / I.DR-prime	Deterministic source replay after a matched prefix.
I.DeferredNBHole	Carried unresolved shared non-blocking decision with NoDependentUse.
I.3	Finite process-local preparation script.
I.4	Construction of the J.PCert records.

12.3 Appendix J

Name	Role
J.LocalGWR	Complete local certificate package; no global replay assumption.
J.DepJ / DepComplete / DepRank / DepAcyclic	Generate, complete, rank, and prove well-founded the construction dependencies.
Finite ideals / J.ValidatedConstructionOrder	Down-closed required-order prefixes used by the J induction; tie ordering among commuting units has no semantic significance.
J.RegPhaseNF	Normalize same-Horizon actions to registered L1/L2/L3 phases.
J.NBDecision	Decide each deferred non-blocking call from the common settled snapshot.
J.UnitExt	Extend one complete Horizon bucket.
J.GWR-Comp	Construct a validated finite-ideal order, reachable J.GWR witness chain, and J.RegPhaseReach from local certificates.
J.1	Top-level observable compatibility theorem.
J.0	Compose an already selected compatible pair.

Name	Role
J.TraceCertCov / J.RefProjCover	Safety-side RTL-prefix certificate coverage, and the optional one-complete-source-run observable-prefix coverage condition. Neither implies K.CertCov.
J.1-Safe	Transfer prefix-closed J.HCanon-invariant safety through Post -> Ref under J.TraceCertCov; one-run use additionally requires J.RefProjCover.
J.HCanon / OfferReach / OfferConf / KObsConf	Build the exact equal-KObs cutpoint bridge.
Corollary J.1	Export equal KObs and reconstructed K-facing facts.

12.4 Appendix K

Name	Role
K.Dead / K.DeadW	Named ordinary and waiver-parameterized reachable drain-closed closed internal wait-cycle predicates.
K.0	Frontier reconstruction from KObs.
K.2 / K.2a	WFG and snapshot drain-closure extensionality.
K.RLAdmReach	Exact source witness is Policy-L admissible: RegPhaseReach + DrainReach.
K.CertifiedCutpoint	Complete package for one explicit (tau_post, nu_post, rho_post) prefix/normalization/cutpoint triple.
K.CertCov	Total package coverage over every reachable proof-relevant prefix/normalization/cutpoint triple.
A5-L	No admissible Policy-L prefix reaches Dead_K.
SDet	One fully specified deterministic Policy-S simulator/run.
K.RespCov / RespClean / CompleteS	Total bounded-response evidence over the complete selected run.
K.2b / K.2c	Proposed serial-dominance and A5-S -> A5-L reduction; K.2b.Q treats only the process-facing blocking serial prefix as committed and excludes failed NB polls from that claim; pending mechanization.
K.1A / K.1A-Total / K.1	One-cutpoint and universal A5-L transfer; the user-facing Policy-S discharge form of K.1 retains proposed status pending mechanization.

12.5 Appendix L

Name	Role
L.1	Causal witness selector for epsilon-modeled source choices.
L.2	Conditional coverage/causality/noninterference theorem under which a realizable snoop-selected witness proves WC.
L.3	NB-CF identity witness; no snooping required for irrelevant failed polls.
L.4	Cadence-sensitive polling requires isolation/snoop alignment.

Name	Role
L.Dir	Ordinal-preserving source consumption of a free-running RTL snoop stream; benign cycle skew is allowed, while order/kind/payload divergence means witness non-existence and no theorem-backed guarantee.
L.5	Online harness realization of an existing abstract witness selector, with W1-W5, L.Dir, coverage, and clean failure reporting as separate obligations.
W1-W5	Recorded ordinal observation, source-side commit gating, registered sampling, comparison/watchdog with proved-versus-inconclusive timeout treatment, and separate coverage proof.

12.6 One-sentence architecture summary

SUMMARY: The proof stack in one sentence

Appendix G defines the observable and operational contract, with Appendix C defining ReturnedAtCycle; Appendix I proves finite process-local replay/preparation under a fixed witness; Appendix J derives a globally reachable source witness through a validated finite-ideal construction and proves trace-compatible safety from local certificates, with J.TraceCertCov governing safety coverage; Appendix L conditionally realizes the witness when latency-sensitive epsilon choices require implementation-aligned selection; and Appendix K uses J's equal-KObs cutpoint plus separate admissibility, proof-relevant normalization/cutpoint coverage, and A5-L evidence to exclude Definition K.Dead/K.DeadW structures in covered RTL cutpoints, with a proposed deterministic Policy-S route for discharging A5-L on its supported fragment.

Source note: theorem and definition names follow the 24 July 2026 scheduling-rules document. This companion intentionally simplifies prose but does not replace the normative statements, side conditions, or proof text.